In [1]:

```
"""
データまたはデータカタログのタイトル・説明文から地理空間情報の
geonamesの候補を推定するための関数群を定義
"""
```

Out[1]:

'¥nデータまたはデータカタログのタイトル・説明文から地理空間情報の¥ngeonamesの候補
を推定するための関数群を定義¥n'

In [2]:

```
# Modules
import geocoder
import requests
import json
import pandas as pd
import chardet

# geocoderのgeoname関連マニュアル
# https://geocoder.readthedocs.io/providers/GeoNames.html

# 独自モジュール
import pick_locationname
```

In [3]:

```
# 各種パラメータの設定
GEONAME_USER_KEY = 'jsugawa'
GEONAME_MAXROWS = 10
GEONAME_LANG = 'ja'
GEONAME_COUNTRY = 'JP'
GEONAME_FEATURECLASS = ['A','P']
GEONAME_KEYWORDSEARCH_URL = 'http://api.geonames.org/searchJSON'
GEONAME_KEYWORDSEARCH_STYLE = 'FULL'
GEONAME_GETFEATURE_URL = 'http://api.geonames.org/getJSON'
GEONAME_HIERARCHY_URL = 'http://api.geonames.org/hierarchyJSON'
GEONAME_FINDNEARBY_URL = 'http://api.geonames.org/findNearbyJSON'
GEONAME_FINDNEARBY_RADIUS = 20
```

In [4]:

```
# 関数の定義

# テキストから地名のキーワードを決める関数を定義(現状は自治体名、都道府県名のみ抽出可能)
def extract_LocationKeywords(text):
    keyword_lg_list = pick_locationname.included_localgovs(text)
    keyword_pf_list = pick_locationname.included_prefectures(text)

    if len(keyword_lg_list) > 0:
        keyword_list = keyword_lg_list
    elif len(keyword_pf_list) > 0:
        keyword_list = keyword_pf_list
    else:
        keyword_list = []

    return keyword_list


# ファイルから緯度経度の一覧を取得する
def extract_lnglatArray_csv(filepath):
    """
    CSVファイルを読み込んで緯度経度のarrayを出力する
    """
    # CSVファイルをDataframeとして読み込む
    try:
        df = pd.read_csv(filepath,encoding='shift_jis')
    except UnicodeDecodeError:
        try :
            df = pd.read_csv(filepath,encoding='utf-8')
        except UnicodeDecodeError:
            print("Error in read_csv")
            return None

    #with open(sample_file_path, "rb") as f:
    #    encoding_sample = chardet.detect(f.read())
    #    print(encoding_sample)

    if ('緯度' in df.columns) and ('経度' in df.columns):
        # 緯度または経度に欠損がある行は削除
        df = df.dropna(subset=['緯度', '経度'])

        # 緯度、経度のnumpy arrayを取得

        lat_array = df['緯度'].values
        lng_array = df['経度'].values

        return lng_array,lat_array
    elif ('latitude' in df.columns) and ('longitude' in df.columns):

        # 緯度または経度に欠損がある行は削除
        df = df.dropna(subset=['latitude', 'longitude'])

        # 緯度、経度のnumpy arrayを取得

        lat_array = df['latitude'].values
        lng_array = df['longitude'].values

        return lng_array,lat_array
    else:
        print("Error: 緯度、経度の列がありません")
```

```
        return None

# ファイルから緯度経度のBoundingboxを取得する
def extract_lnglatBbox_csv(filepath):

    lng_array,lat_array = extract_lnglatArray_csv(filepath)
    east = lng_array.max()
    west = lng_array.min()
    north = lat_array.max()
    south = lat_array.min()

    return east,west,north,south



# 地名キーワードからgeonameを検索する関数を定義(未使用とする)
def FindByKeyword(
    locationKeyword,
    key=GEONAME_USER_KEY,
    maxRows=GEONAME_MAXROWS,
    featureClass=[],
    lang=GEONAME_LANG,
    country=GEONAME_COUNTRY):
    """
    地名のキーワードを指定して、全属性を対象に検索し、候補となるgeonameのリストを出力
    """
    g = geocoder.geonames(
        location=locationKeyword,
        key=key,
        maxRows=maxRows,
        featureClass=featureClass,
        lang=lang,
        country=country,
        )

    return g


# 地名キーワードからgeonameを検索する関数を定義
def FindByKeyword_name(
    locationKeyword,
    key=GEONAME_USER_KEY,
    maxRows=GEONAME_MAXROWS,
    featureClass=[],
    lang=GEONAME_LANG,
    country=GEONAME_COUNTRY,
    style=GEONAME_KEYWORDSEARCH_STYLE):
    """
    地名のキーワードを指定して、name属性を対象に検索し、
    候補となるgeonameを表すdictonaryのリストを出力
    """


    params ={
    'name':locationKeyword,
    'username':key,
    'maxRows':maxRows,
    'featureClass':featureClass,
    'lang':lang,
    'country':country,
    'style':style
    }
```

```
    r = requests.get(
        url=GEONAME_KEYWORDSEARCH_URL,
        params=params
        )

    #print('request query: ',r.url)
    #print('status code: ', r.status_code)

    result_dict = json.loads(r.text)
    return result_dict['geonames']


# geoname idから属性を調べる関数を定義
def getFeautures(geonameId, key=GEONAME_USER_KEY, lang=GEONAME_LANG):
    """
    指定したgeonameIdの地名の属性値を辞書形式で出力する
    """
    params ={
        'geonameId':geonameId,
        'username':key,
        'lang':lang,
    }

    r = requests.get(
        url=GEONAME_GETFEATURE_URL,
        params=params
        )

    #print('request query: ',r.url)
    #print('status code: ', r.status_code)

    result_dict = json.loads(r.text)
    return result_dict


# geonameIdからHierarchy(上位の地名)を調べる関数を定義
def getHierarchyList(geonameId, key=GEONAME_USER_KEY, lang=GEONAME_LANG):
    """
    指定したgeonameIdのHierarchy(上位の地名)の一覧をリストで出力する
    """
    params ={
        'geonameId':geonameId,
        'username':key,
        'lang':lang,
    }

    r = requests.get(
        url=GEONAME_HIERARCHY_URL,
        params=params
        )

    #print('request query: ',r.url)
    #print('status code: ', r.status_code)

    result_dict = json.loads(r.text)
    # return result_dict['geonames']

    list_hierarychy=[]
    for elem in result_dict['geonames']:
        list_hierarychy.append(elem['name'])
```

```
    return list_hierarychy


# geonameIdから上位の地名を含むFullnameを出力する関数を定義
def getFullname(geonameId, key=GEONAME_USER_KEY, lang=GEONAME_LANG, mode='Country'):
    """
    指定したgeonameIdのFullnameを出力する
    (例: 日本>千葉県>千葉市>中央区)
    """
    list = getHierarchyList(geonameId)

    if mode == 'Continent': # 例 アジア>日本>千葉県>千葉市>中央区
        return '>'.join(list[1:])
    elif mode == 'Country': # 例 日本>千葉県>千葉市>中央区
        return '>'.join(list[2:])
    elif mode == 'Prefecture': # 例 千葉県>千葉市>中央区
        return '>'.join(list[3:])
    elif mode == 'Full': # 例 Earth>アジア>日本>千葉県>千葉市>中央区
        return '>'.join(list[0:])
    else:
        return '>'.join(list)



# 緯度経度のBoundingBox範囲から検索する
# (FeatureClass='A' or 'P'のみ抽出)
def FindbyBbox(
    east, west, north, south,
    key=GEONAME_USER_KEY,
    maxRows=GEONAME_MAXROWS,
    featureClass=GEONAME_FEATURECLASS,
    lang=GEONAME_LANG,
    country=GEONAME_COUNTRY,
    style=GEONAME_KEYWORDSEARCH_STYLE):

    params ={
    'q':"",
    'username':key,
    'maxRows':maxRows,
    'featureClass':featureClass,
    'lang':lang,
    'country':country,
    'style':style,
    'east':east,
    'west':west,
    'north':north,
    'south':south,
    }

    r = requests.get(
        url=GEONAME_KEYWORDSEARCH_URL,
        params=params
        )

    #print('request query: ',r.url)
    #print('status code: ', r.status_code)

    result_dict = json.loads(r.text)
    return result_dict['geonames']
```

```
def FindbyBbox_old(east, west, north, south):
    g = geocoder.geonames(
        '',
        key=GEONAME_USER_KEY,
        maxRows=GEONAME_MAXROWS,
        featureClass=GEONAME_FEATURECLASS,
        lang=GEONAME_LANG,
        country=GEONAME_COUNTRY,
        east=east,
        west=west,
        north=north,
        south=south,
        )

    return g


# 手動でFindNearby用の関数を定義(geocoderのProximity指定だと結果がいまいちだったため)
def FindNearby(
    lng,lat,
    key=GEONAME_USER_KEY,
    radius=GEONAME_FINDNEARBY_RADIUS,
    maxRows=GEONAME_MAXROWS,
    featureClass=GEONAME_FEATURECLASS,
    lang=GEONAME_LANG,
    country=GEONAME_COUNTRY,
    style=GEONAME_KEYWORDSEARCH_STYLE
    ):
    """
    指定した緯度、経度の近くの地名をリスト形式で出力する
    """
    params ={
        'lng':lng,
        'lat':lat,
        'username':key,
        'radius':radius,
        'maxRows':maxRows,
        'featureClass':featureClass,
        'lang':lang,
        'country':country,
        'style':style,
    }

    r = requests.get(
        url=GEONAME_FINDNEARBY_URL,
        params=params
        )

    # print('request query: ',r.url)
    # print('status code: ', r.status_code)

    result_dict = json.loads(r.text)
    return result_dict['geonames']
```

In [5]:

```
# 関数の動作テスト
if __name__ == '__main__':
    # 関数extract_LocationKeywordsのテスト1
    print("==== extract_LocationKeywordsのテスト1 ====")
    title_text = "【蕨市】避難施設一覧"
    description_text = "蕨市内の避難施設の一覧です。"
    # https://opendata.pref.saitama.lg.jp/data/dataset/c-users-2348-desktop
    print("title:", title_text)
    print("description:", description_text)
    print("Keywords:",extract_LocationKeywords(title_text + description_text))

    # 関数extract_LocationKeywordsのテスト2
    print("==== extract_LocationKeywordsのテスト2 ====")
    title_text = "【埼玉県】公共施設情報"
    description_text = "埼玉県が保有する公共施設の住所、連絡先及び位置情報等のデータ"
    # https://opendata.pref.saitama.lg.jp/data/dataset/a2290sisetu
    print("title:", title_text)
    print("description:", description_text)
    print("Keywords:",extract_LocationKeywords(title_text + description_text))
    print("¥n")
```

```
==== extract_LocationKeywordsのテスト1 ====
title: 【蕨市】避難施設一覧
description: 蕨市内の避難施設の一覧です。
Keywords: ['蕨市']
==== extract_LocationKeywordsのテスト2 ====
title: 【埼玉県】公共施設情報
description: 埼玉県が保有する公共施設の住所、連絡先及び位置情報等のデータ
Keywords: ['埼玉県']
```

In [6]:

```
    # extract_latlngArray_csv関数のテスト
    print("==== extract_latlngArray_csvのテスト ====")
    sample_file_path = 'sample/warabi_hinansisetu.csv'
    lng_array_sample, lat_array_sample,  = extract_lnglatArray_csv(filepath=sample_file_path)
    print("file:", sample_file_path)

    print("longitude array:", lng_array_sample)
    print("latitude array:",lat_array_sample)
    print("¥n")


    # extract_latlngBbox_csv関数のテスト
    print("==== extract_latlngBbox_csvのテスト ====")
    sample_file_path = 'sample/warabi_hinansisetu.csv'
    east_sample, west_sample, north_sample, south_sample = extract_lnglatBbox_csv(filepath=sampl
e_file_path)
    print("file:", sample_file_path)
    print("east, west, north, south:",east_sample, west_sample, north_sample, south_sample)
    print("¥n")
```

```
==== extract_latlngArray_csvのテスト ====
file: sample/warabi_hinansisetu.csv
longitude array: [139.6777    139.6763611 139.6768878 139.6756306 139.6712556 139.
6838
 139.6820083 139.6774167 139.6943306 139.6906711 139.6869056 139.6852139
 139.68665   139.6909667 139.6913806 139.6955028 139.7009028 139.6994111
 139.7020778 139.7026167 139.704475  139.7035667 139.7078028 139.7060167
 139.6886544 139.681     139.6843    139.708    ]
latitude array: [35.82070833 35.81930833 35.82445639 35.82534444 35.82696667 35.82
767778
 35.82341389 35.82967222 35.82363333 35.82146528 35.82082222 35.8226
 35.81826667 35.81517778 35.81342222 35.81494167 35.81787222 35.81504444
 35.81626944 35.82188611 35.82177778 35.82026111 35.82109722 35.82259722
 35.81621417 35.8259     35.8231     35.8196    ]


==== extract_latlngBbox_csvのテスト ====
file: sample/warabi_hinansisetu.csv
east, west, north, south: 139.708 139.6712556 35.82967222 35.81342222
```

In [7]:

```
# FindByKeyword動作テスト1
print("==== FindByKeywordのテスト1 ====")
locationKeyword_sample1 = '羽田空港'
g1 = FindByKeyword(locationKeyword_sample1)
print("Keyword: ", locationKeyword_sample1)
for elem in g1:
    print(
        elem.geonames_id,
        elem.address,
        elem.country,
        elem.state,
        elem.lat,
        elem.lng,
        elem.feature_class, )
print("¥n")

# FindByKeyword動作テスト2
print("==== FindByKeywordのテスト1 ====")
locationKeyword_sample2='中央区' # 中央区
g2 = FindByKeyword(locationKeyword_sample2,featureClass=['A','P'])
print("Keyword: ", locationKeyword_sample2)
for elem in g2:
    print(
        elem.geonames_id,
        elem.address,
        elem.country,
        elem.state,
        elem.lng,
        elem.lat,
        elem.feature_class, )
print("¥n")
```

```
==== FindByKeywordのテスト1 ====
Keyword:  羽田空港
6300412 羽田空港 日本 東京都 35.55226 139.77969 S
10957423 羽田空港 日本 東京都 35.55281 139.77933 L


==== FindByKeywordのテスト1 ====
Keyword:  中央区
1859171 神戸 日本 兵庫県 135.183 34.6913 P
2113015 千葉市 日本 千葉県 140.11667 35.6 P
1858421 熊本 日本 熊本県 130.69181 32.80589 P
1848254 Yono 日本 埼玉県 139.63333 35.88333 P
1864487 中央区 日本 東京都 139.77705 35.66993 A
1863623 銀座 日本 東京都 139.76717 35.67184 P
1849687 月島 日本 東京都 139.78195 35.66259 P
8534447 中央区 日本 千葉県 140.12941 35.58434 A
7391689 中央区 日本 大阪府 135.51413 34.68011 A
8534438 中央区 日本 埼玉県 139.62468 35.88133 A
```

In [8]:

```
# FindByKeyword_name動作テスト1
print("==== FindByKeyword_nameのテスト ====")
locationKeyword_sample1 = '羽田空港'
result = FindByKeyword_name(locationKeyword_sample1)
print("Keyword: ", locationKeyword_sample1)
print("Result Count: ", len(result))
print("Result:")

for elem in result:
    print(
        elem['geonameId'],
        elem['countryName'],
        elem['adminName1'],
        elem['adminName2'],
        elem['name'],
        elem['lng'],
        elem['lat'],
        elem['fcl']
    )

#print(result)
print("¥n")
# FindByKeyword_name動作テスト2
print("==== FindByKeyword_nameのテスト ====")
locationKeyword_sample2 = '中央区'
result = FindByKeyword_name(locationKeyword_sample2)
print("Keyword: ", locationKeyword_sample2)
print("Result Count: ", len(result))
print("Result:")

for elem in result:
    print(
        elem['geonameId'],
        elem['countryName'],
        elem['adminName1'],
        elem['adminName2'],
        elem['name'],
        elem['lng'],
        elem['lat'],
        elem['fcl']
    )

#print(result)
print("¥n")
```

```
==== FindByKeyword_nameのテスト ====
Keyword:  羽田空港
Result Count:  2
Result:
6300412 日本 東京都 大田区 羽田空港 139.77969 35.55226 S
10957423 日本 東京都  羽田空港 139.77933 35.55281 L


==== FindByKeyword_nameのテスト ====
Keyword:  中央区
Result Count:  10
Result:
1864487 日本 東京都 中央区 中央区 139.77705 35.66993 A
8693272 日本 神奈川県 相模原市 中央区 139.36659 35.5625 A
8534438 日本 埼玉県 さいたま市 中央区 139.62468 35.88133 A
8534447 日本 千葉県 千葉市 中央区 140.12941 35.58434 A
7391689 日本 大阪府 大阪市 中央区 135.51413 34.68011 A
8739908 日本 熊本県 熊本市 中央区 130.7234 32.79968 A
7371146 日本 兵庫県 神戸市 中央区 135.19803 34.6902 A
11828195 日本 北海道  中央区 140.11725 41.80131 P
7468171 日本 新潟県 新潟市 中央区 139.04846 37.90403 A
7429523 日本 北海道 札幌市 中央区 141.30209 43.0384 A
```

In [9]:

```python
# GetFeatures関数の動作テスト
print("==== GetFeaturesのテスト ====")

geonameId_sample=8534447
result_elem = getFeautures(geonameId=geonameId_sample)

print("geonameId:", geonameId_sample)
print(
    result_elem['geonameId'],
    result_elem['name'],
    result_elem['countryName'],
    result_elem['adminName1'],
    result_elem['adminName2'],
    result_elem['lng'],
    result_elem['lat'],
    result_elem['fcl'],
    result_elem['bbox']
)
print("¥n")
```

```
==== GetFeaturesのテスト ====
geonameId: 8534447
8534447 中央区 日本 千葉県 千葉市 140.12941 35.58434 A {'east': 140.175437562, 'so
uth': 35.542526333000005, 'north': 35.628887667, 'west': 140.081593606, 'accuracyL
evel': 0}
```

In [10]:

```
# getHierarchyList関数の動作テスト
print("==== getHierarchyListのテスト ====")

geonameId_sample=8534447
result = getHierarchyList(geonameId=geonameId_sample)

print("geonameId:", geonameId_sample)
print(result)
```

```
==== getHierarchyListのテスト ====
geonameId: 8534447
['Earth', 'アジア', '日本', '千葉県', '千葉市', '中央区']
```

In [11]:

```
# getFullname関数の動作テスト
print("==== getFullnameのテスト1 ====")

geonameId_sample=8534447 # 千葉県>千葉市>中央区
for mode_sample in ["Full", "Continent", "Country", "Prefecture"]:
    result = getFullname(geonameId=geonameId_sample,mode=mode_sample)
    print("geonameId:", geonameId_sample, "mode:", mode_sample)
    print(result)

print("==== getFullnameのテスト2 ====")
geonameId_sample=6930927 # 札幌駅

for mode_sample in ["Full", "Continent", "Country", "Prefecture"]:
    result = getFullname(geonameId=geonameId_sample,mode=mode_sample)
    print("geonameId:", geonameId_sample, "mode:", mode_sample)
    print(result)

print("==== getFullnameのテスト3 ====")
geonameId_sample=1853226 # 埼玉県

for mode_sample in ["Full", "Continent", "Country", "Prefecture"]:
    result = getFullname(geonameId=geonameId_sample,mode=mode_sample)
    print("geonameId:", geonameId_sample, "mode:", mode_sample)
    print(result)

print("¥n")
```

```
==== getFullnameのテスト1 ====
geonameId: 8534447 mode: Full
Earth>アジア>日本>千葉県>千葉市>中央区
geonameId: 8534447 mode: Continent
アジア>日本>千葉県>千葉市>中央区
geonameId: 8534447 mode: Country
日本>千葉県>千葉市>中央区
geonameId: 8534447 mode: Prefecture
千葉県>千葉市>中央区
==== getFullnameのテスト2 ====
geonameId: 6930927 mode: Full
Earth>アジア>日本>北海道>札幌市>北区>札幌駅
geonameId: 6930927 mode: Continent
アジア>日本>北海道>札幌市>北区>札幌駅
geonameId: 6930927 mode: Country
日本>北海道>札幌市>北区>札幌駅
geonameId: 6930927 mode: Prefecture
北海道>札幌市>北区>札幌駅
==== getFullnameのテスト3 ====
geonameId: 1853226 mode: Full
Earth>アジア>日本>埼玉県
geonameId: 1853226 mode: Continent
アジア>日本>埼玉県
geonameId: 1853226 mode: Country
日本>埼玉県
geonameId: 1853226 mode: Prefecture
埼玉県
```

In [12]:

```
# FindByBbox_old関数のテスト
print("==== FindByBbox_oldのテスト ====")
east_sample=141.0
west_sample=139.9
north_sample=35.8
south_sample=35.4
g3 = FindbyBbox_old(east=east_sample,west=west_sample,north=north_sample,south=south_sample)
print("east,west,north,south:", east_sample,west_sample,north_sample,south_sample)
for elem in g3:
    print(
        elem.geonames_id,
        elem.address,
        elem.country,
        elem.state,
        elem.lng,
        elem.lat,
        elem.feature_class, )
print("¥n")
```

```
==== FindByBbox_oldのテスト ====
east,west,north,south: 141.0 139.9 35.8 35.4
2113015 千葉市 日本 千葉県 140.11667 35.6 P
1863905 本町 日本 千葉県 139.98648 35.70129 P
2113014 千葉県 日本 千葉県 140.12333 35.60506 A
2111684 成田 日本 千葉県 140.31667 35.78333 P
2112664 市原市 日本 千葉県 140.08333 35.51667 P
1857553 松戸市 日本 千葉県 139.90144 35.77995 P
2110579 八街市 日本 千葉県 140.31667 35.65 P
2111220 佐倉 日本 千葉県 140.23333 35.71667 P
2111855 モバラ 日本 千葉県 140.29608 35.42583 P
2110480 四街道市 日本 千葉県 140.16667 35.65 P
```

In [13]:

```
# FindByBbox関数のテスト
print("==== FindByBboxのテスト ====")
east_sample=141.0
west_sample=139.9
north_sample=35.8
south_sample=35.4

result = FindbyBbox(east=east_sample,west=west_sample,north=north_sample,south=south_sample)
print("east,west,north,south:", east_sample,west_sample,north_sample,south_sample)

print("Result Count: ", len(result))
print("Result:")

for elem in result:
    print(
        elem['geonameId'],
        elem['countryName'],
        elem['adminName1'],
        elem['adminName2'],
        elem['name'],
        elem['lng'],
        elem['lat'],
        elem['fcl']
    )

#print(result)
print("¥n")
```

```
==== FindByBboxのテスト ====
east,west,north,south: 141.0 139.9 35.8 35.4
Result Count:  10
Result:
2113015 日本 千葉県 千葉市 千葉市 140.11667 35.6 P
1863905 日本 千葉県 船橋市 本町 139.98648 35.70129 P
2113014 日本 千葉県  千葉県 140.12333 35.60506 A
2111684 日本 千葉県 成田市 成田 140.31667 35.78333 P
2112664 日本 千葉県 市原市 市原市 140.08333 35.51667 P
1857553 日本 千葉県 松戸市 松戸市 139.90144 35.77995 P
2110579 日本 千葉県 八街市 八街市 140.31667 35.65 P
2111220 日本 千葉県 佐倉市 佐倉 140.23333 35.71667 P
2111855 日本 千葉県 茂原市 モバラ 140.29608 35.42583 P
2110480 日本 千葉県 四街道市 四街道市 140.16667 35.65 P
```

In [14]:

```
# FindNearby関数の動作テスト
print("==== FindNearbyのテスト ====")
lng_sample = 139.46125
lat_sample = 35.70552

result = FindNearby(lng=lng_sample,lat=lat_sample)
print("lng,lat:", lng_sample,lat_sample)

for elem in result:
    print(
        elem['geonameId'],
        elem['countryName'],
        elem['adminName1'],
        elem['adminName2'],
        elem['name'],
        elem['lng'],
        elem['lat'],
        elem['fcl'],
        elem['distance']
    )

print("¥n")
```

```
==== FindNearbyのテスト ====
lng,lat: 139.46125 35.70552
1858962 日本 東京都 国分寺市 国分寺市 139.46125 35.70552 A 0.00024
1859265 日本  Kita-tama-gun Kita-tama-gun 139.45 35.71667 A 1.60187
1858346 日本 東京都 国立市 国立市 139.43878 35.68634 A 2.94403
1859115 日本 東京都 小平市 小平市 139.48173 35.72738 A 3.05232
1851419 日本   Sunagawa-machi 139.41667 35.71667 A 4.21972
1859087 日本 東京都 小金井市 小金井市 139.51104 35.70108 A 4.53318
1864153 日本 東京都  Fuchū-machi 139.48333 35.66667 A 4.75181
1862769 日本 東京都  Higashi-murayama-machi 139.46667 35.75 A 4.9595
1851306 日本 東京都 立川市 立川市 139.40453 35.71447 A 5.22787
1864148 日本 東京都 府中市 府中市 139.5 35.66667 A 5.55764
```

In [15]:

```
    # データカタログのタイトル、説明文から地名キーワードを抽出し、地名候補を出力
    print("==== データカタログのタイトル、説明文から地名キーワードを抽出し、地名候補を出力 ====")
)
    title_sample = "八王子市の赤ちゃん・ふらっと設置施設"
    description_sample = "【八王子市】子育て関連オープンデータ"
    # https://catalog.data.metro.tokyo.lg.jp/dataset/t132012d0000000031

    locationKeyword_list = extract_LocationKeywords(title_sample+description_sample)
    locationKeyword_sample = locationKeyword_list[0]

    result = FindByKeyword_name(locationKeyword_sample)
    print("title:",title_sample)
    print("description:", description_sample)
    print("Keyword:", locationKeyword_sample)

    for elem in result:
        print(
            elem['geonameId'],
            elem['countryName'],
            elem['adminName1'],
            elem['adminName2'],
            elem['name'],
            elem['lng'],
            elem['lat'],
            elem['fcl']
        )

    #print(result)
    print("¥n")
```

```
==== データカタログのタイトル、説明文から地名キーワードを抽出し、地名候補を出力 ==
==
title: 八王子市の赤ちゃん・ふらっと設置施設
description: 【八王子市】子育て関連オープンデータ
Keyword: 八王子市
1863440 日本 東京都 八王子市 八王子 139.32389 35.65583 P
8304375 日本 東京都 八王子市 八王子市 139.28766 35.65983 A
```

In [16]:

```
    # ファイルからboundingboxを算出し、そのエリアにある地名候補を出力
    print("==== ファイル取得から地名候補出力までのテスト ====")
    sample_file_path = 'sample/warabi_hinansisetu.csv'
    east_sample, west_sample, north_sample, south_sample = extract_lnglatBbox_csv(filepath=sampl
e_file_path)
    result = FindbyBbox(east=east_sample,west=west_sample,north=north_sample,south=south_sample)
    print("File:", sample_file_path)
    print("east,west,north,south:", east_sample,west_sample,north_sample,south_sample)

    print("Result Count: ", len(result))
    print("Result:")

    for elem in result:
        print(
            elem['geonameId'],
            elem['countryName'],
            elem['adminName1'],
            elem['adminName2'],
            elem['name'],
            elem['lng'],
            elem['lat'],
            elem['fcl']
        )

    #print(result)
    print("¥n")
```

```
==== ファイル取得から地名候補出力までのテスト ====
File: sample/warabi_hinansisetu.csv
east,west,north,south: 139.708 139.6712556 35.82967222 35.81342222
Result Count:  10
Result:
1907301 日本 埼玉県 戸田市 シモトダ 139.6853 35.815 P
1848902 日本 埼玉県 蕨市 蕨市 139.6855 35.82526 A
8572571 日本 埼玉県 蕨市 きたまち 139.68181 35.82935 P
8572572 日本 埼玉県 蕨市 にしきちょう 139.67181 35.82545 P
11032606 日本 埼玉県 戸田市 カミトダ 139.67576 35.81706 P
11032713 日本 埼玉県 蕨市 つかごし 139.69996 35.82378 P
11032714 日本 埼玉県 蕨市 ミナミチョウ 139.69442 35.81807 P
11032743 日本 埼玉県 川口市 しばしんまち 139.69558 35.82938 P
11032744 日本 埼玉県 川口市 芝中田 139.70099 35.82935 P
11612339 日本 埼玉県 蕨市 蕨 139.68545 35.82188 P
```